LAPORAN TUGAS BESAR 1

IF3070 - DASAR INTELEGENSI DAN ARTIFISIAL
KELAS K2

IMPLEMENTASI ALGORITMA LOCAL SEARCH DALAM OPTIMASI PENEMPATAN KENDARAAN DALAM KAPAL



Disusun oleh:

Zafran Abqary Faisal - 18225084

Padmanabha Adyaksa Setyanto - 18222142

Fawwaz Aydin Mustofa - 18222109

TAHUN 2026/2027

A. Deskripsi Persoalan

1. Latar belakang

Pada studi kasus ini, kelompok kami berperan sebagai Forward Deployed Engineer (FDE) di PT ~~Owen~~ Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan telah menerima kontak pengiriman kendaraan yang berbeda-beda dari berbagai dealer. Setiap kendaraan memiliki spesifikasi yang berbeda-beda, seperti ukuran, dimensi, berat, tarif, dan waktu kedatangan kendaraan.

Oleh karena itu, Kami ditugaskan untuk membangun sistem yang dapat mengatur penempatan kendaraan dalam kapal secara otomatis. Sistem bertujuan agar penempatan kendaraan dapat dilakukan dengan optimal untuk menghasilkan keuntungan sebesar mungkin untuk perusahaan. Penempatan kendaraan ini memiliki objektif, constraint, dan tujuan yang harus dicapai

2. Penjelasan Permasalahan

a. Representasi Kapal dan Kendaraan

- Koordinat dimulai dari (0,0) yang berada di pojok kiri atas kapal.
- Kapal direpresentasikan sebagai grid 2 dimensi berbentuk persegi panjang dengan ukuran panjang dan lebar tertentu.
- Setiap kendaraan memiliki dimensi panjang, lebar, berat, tarif pengiriman, dan waktu kedatangan. Kendaraan dapat ditempatkan di dalam ataupun di luar kapal

  .

b. Objektif

Penempatan kendaraan dilakukan agar perusahaan dapat menerima tarif sebanyak mungkin dari pengiriman kendaraan. Nilai total tarif dapat ditingkatkan dengan mengangkut kendaraan sebanyak mungkin dan memprioritaskan kendaraan dengan tarif yang tinggi. Penempatan ini juga harus tidak melanggar constraint yang ada.

c. Constraint

- Seluruh grid yang ditempati kendaraan tidak boleh bertumpang tindih dengan kendaraan lain.
- Seluruh bagian kendaraan yang menaiki kapal harus dalam grid kapal.
- Total berat kendaraan tidak boleh melebihi batas maksimum berat kapal.

B. Pembahasan Algoritma

1. Program Inti

Program ini adalah fondasi dari algoritma local search yang akan dilakukan agar algoritma tersebut dapat berjalan dengan baik. Program ini berisi tentang kelas yang akan digunakan, perhitungan fungsi objektif dan constraint, pembentukan initial state, dan langkah tetangga yang dapat dilakukan.

a. Kelas dan State dari Entitas

- Kelas Vehicle (Kendaraan). Menyimpan atribut:
  - id (identifikasi unik tiap kendaraan)
  - length (panjang kendaraan dalam satuan unit)
  - width (lebar kendaraan dalam satuan unit)
  - fee (tarif untuk pengiriman kendaraan dalam satuan jutaan rupiah)
  - eta (estimasi waktu kedatangan kendaraan dalam hari)

- Kelas Ship (Kapal). Menyimpan atribut:
  - length (panjang kapal dalam satuan unit)
  - width (lebar kapal dalam satuan unit)
  - max_capacity (kapasitas berat maksimum untuk kapal

- Kelas Placement (Posisi kendaraan dalam kapal). Menyimpan atribut:
  - x (posisi x kendaraan dalam koordinat)
  - y (posisi y kendaraan dalam koordinat)
  - orientation (orientasi kendaraan (horizontal atau vertikal)

b. Fungsi Pembantu

Fungsi-fungsi berikut digunakan untuk membantu berjalannya fungsi utama yang akan digunakan nantinya.

- def flip(orientation: str) -> str:
  Mengubah orientasi kendaraan dari horizontal ke vertikal dan sebaliknya.

- def replaced(state: State, changes: Dict[str, Optional[Placement]]) -> State:
  Membuat salinan state untuk perubahan placement kendaraan.

- def swapped(state: State, a: str, b: str) -> State:
  Membuat salinan state untuk menukar posisi dua kendaraan.

- def rotated(state: State, vid: str) -> State:
  Membuat salinan state untuk perubahan orientasi kendaraan.

- def footprint(v: Vehicle, orientation: str) -> tuple[int, int]:
  Melihat ukuran kendaraan dalam x dan y sesuai orientation.
  Contoh: vehicle w=3, l=4 menghasilkan (3, 4) jika Vertical dan (4, 3) jika Horizontal.

- def cells(v: Vehicle, p: Placement) -> list[tuple[int, int]]:
  Mendaftar semua grid yang ditempati kendaraan.

c. **Fungsi Objektif dan Constraint**
   Fungsi ini dibuat untuk menentukan objektif dan constraint dari permasalahan yang akan dibuat. Fungsi ini dan selanjutnya disimpan dalam kelas Problem.

   - def objective(self, state: State) -> int:
     Menghitung nilai total fee dari tiap kendaraan dalam kapal.

   - def total_weight(self, state: State) -> int:
     Menghitung nilai total weight kendaraan di atas kapal.

d. **Fungsi Initial State**

   - def from_json(cls, path: str) -> "Problem":
     Membaca data .json yang berisi tentang data setiap kendaraan dan kapal.

   - def is_valid(self, state: State) -> bool:
     Mengecek apakah penempatan kendaraan di atas kapal valid atau tidak.

   - def random_placement
     Membuat placement acak untuk kendaraan tertentu.

   - def random_state(self, rng: random.Random, attempts: int = 20) -> State:
     Membuat initial state yang valid.

e. **Fungsi Langkah Neighbor**
   Fungsi-fungsi selanjutnya digunakan untuk mempermudah berjalannya algoritma local search yang akan dijalankan.

2. Algoritma Hill-Climbing (with Sideways Move)

Hill-Climbing Search adalah algoritma *local search* yang dimulai dari *state* awal acak, yang berpindah ke *state* tetangga yang memiliki *value* yang lebih tinggi. Algoritma ini tidak mengingat memori pencarian dan terus berpindah ke *state* terbaik. Pencarian dihentikan jika tidak ada tetangga yang memiliki *value* yang lebih tinggi. Sebagai Analogi, pencarian ini diibaratkan sebagai mendaki Gunung Everest dengan kabut tebal dengan amnesia. Namun, Hill-Climbing memiliki kekurangan dimana pencarian bisa terjebak di *local optimum*, yaitu kondisi jika *state* memiliki *value* lebih tinggi dibandingkan *state* tetangganya, sehingga pencarian berhenti disana, meskipun masih ada *state* dengan *value* yang lebih tinggi.

Hill-Climbing Search with Sideways Move adalah variasi Hill-Climbing yang memperbolehkan pergerekan ke tetangga dengan *value* yang sama. Metode ini membuat agen dapat berpindah dengan lebih leluasa dan mencegahnya terjebak di *local optimum.*

Berikut rincian algoritma Hill-Climbing with Sideways Move dalam program ini.

1. Algoritma dimulai dengan menerima parameter dari kelas Problem dari file Fondasi.

2. Membuat *initial state* acak, dan nilai *value*-nya dicatat
3. Algoritma memanggil semua *neighbor state* yang valid, lalu sistem memilih *neighbor state* dengan *value* tertinggi untuk berpindah.
4. Jika tidak ada *neighbor state* dengan *value* yang lebih tinggi, sistem diperbolehkan melakukan *sideways move* ke *state* dengan nilai *value* yang sama.
5. Algoritma pencarian berhenti dalam kondisi berikut:
    a. Tidak ada tetangga yang valid tersisa (*no_neighbor*).
    b. Semua nilai tetangga lebih kecil (*local_optimum*).
    c. Jumlah pergerakan *sideways* melebihi batas *max_sideways* yang diberikan (*max_sideways*).
    d. Jumlah iterasi mencapai jumlah maksimum (*max_iteration*).

Algorithm akan mengembalikan objek searchResult yang menunjukkan *state* awal dan akhir, nilai objektif awal dan akhir, riwayat nilai, waktu komputasi, jumlah iterasi yang dilakukan, dan alasan pencarian berhenti.

3. Simulated Annealing

Simulated Annealing adalah algoritma *local search* yang mencari solusi optimal dengan sesekali menerima langkah yang lebih buruk, supaya pencarian tidak terjebak di *local optimum* seperti Hill-Climbing

Cara Kerja

- Mulai dari state awal acak dan suhu awal T_0 yang tinggi
- Setiap iterasi, pilih satu neighbor secara acak, lalu hitung ΔE = nilai neighbor − nilai current
- Tentukan apakah neighbor diterima:
    - ΔE > 0 selalu diterima
    - ΔE < 0 diterima dengan probabilitas P = e^(ΔE/T).
- Turunkan suhu
- Ulangi sampai T < T_min atau batas iterasi tercapai. State terbaik yang pernah ditemukan menjadi hasil akhir

Karena ΔE bernilai negatif untuk langkah buruk, nilai e^(ΔE/T) selalu berada di antara 0 dan 1:

- Semakin buruk langkahnya (ΔE makin negatif), semakin kecil peluang diterima.
- Semakin tinggi suhunya, semakin besar peluang diterima.

Berikut adalah implementasi Simulated Annealing:

- Fungsi simulated_annealing()
    - Inisialisasi. Buat initial state acak, lalu hitung nilainya (total ShippingFee). Simpan sebagai current dan best. Suhu awal T = 100
- Loop selama T > 0,01 dengan maksimum iterasi = 10.000. Pada setiap iterasi:
    - Ambil satu neighbor acak. `random_neighbor()` dari `Fondasi.py` memilih acak salah satu dari tiga langkah (tukar dua kendaraan, pindah posisi, atau rotasi) dan hanya mengembalikan neighbor yang valid. Jika gagal

menemukan neighbor valid, hasilnya None dan iterasi dilewati tanpa perubahan state.
    - Hitung ΔE = `nilai neighbor - nilai current`. Karena masalah ini maksimasi, ΔE > 0 berarti neighbor lebih baik
    - Putuskan apakah neighbor diterima:
        - ΔE > 0: selalu diterima.
        - ΔE = 0: diterima (*sideways move*, setara dengan $e^0 = 1$)
        - ΔE < 0: hitung `prob = e^(ΔE/T)`, catat ke `prob_history`, lalu terima jika angka acak `rng.random()` < `prob`. Langkah buruk yang diterima ditambahkan ke `accepted_worse`.
    - Pindah ke neighbor jika diterima
    - Perbarui best dan hitung stuck, jika current lebih baik dari best, best diperbarui dan `no_improve` di-reset. Jika tidak, `no_improve` bertambah. Saat mencapai `stuck_patience`, `stuck_count` bertambah 1 dan `no_improve` di-reset
    - Catat riwayat dan turunkan suhu. nilai current dan best disimpan untuk plot, lalu `temp *= cooling_rate`
- Kembalikan state terbaik (best), nilainya, durasi, banyak iterasi, frekuensi stuck, dan riwayat untuk plot

Fungsi pendukung

- `state_to_dict()`: mengubah state ke dict agar bisa disimpan ke JSON.
- `deck_to_text()`: menampilkan geladak sebagai grid teks beserta kendaraan di dalam dan di luar kapal.
- `save_result()`: menyimpan hasil run ke file JSON.
- `plot_result()`: membuat plot nilai objective dan e^(ΔE/T) terhadap iterasi.

4. Genetic Algorithm

Algoritma Genetika (*Genetic Algorithm*) adalah teknik pencarian dan optimasi heuristik yang diinspirasi oleh proses seleksi alam dan genetika (Goldberg, 1989). Dalam studi kasus penempatan kendaraan di geladak kapal kargo ini, *Genetic Algorithm* digunakan untuk mencari kombinasi penataan posisi (x, y) dan orientasi kendaraan yang menghasilkan nilai *objective function* (tarif pengiriman / *shipping fee*) maksimum tanpa melanggar batasan (*constraint*) yang berlaku.

Adapun rincian implementasi komponen utama *Genetic Algorithm* dalam program adalah sebagai berikut:

- · **Struktur Kromosom / Individu:**

Setiap individu (kromosom) merepresentasikan satu solusi lengkap penataan seluruh kendaraan (V1 hingga V5) di atas geladak kapal. Kromosom ini terdiri dari array atau list objek penempatan (*placement*) yang menyimpan koordinat posisi x, posisi y, serta orientasi kendaraan (horizontal atau vertikal).

- · **Ukuran Populasi dan Jumlah Generasi (Iterasi)**

Populasi adalah kumpulan sejumlah individu yang dievaluasi pada setiap siklus generasi. Dalam program ini, parameter ukuran populasi divariasikan (seperti 5, 30, dan 40 individu) untuk melihat pengaruh keragaman genetik, serta diuji dengan variasi jumlah iterasi atau generasi (30, 50, dan 100 iterasi) guna mengamati tingkat konvergensi algoritma menuju solusi optimal.

- · **Proses Seleksi (*Selection*), Persilangan (*Crossover*), dan Mutasi (*Mutation*)**
  - **Seleksi:** Dilakukan untuk memilih individu-individu terbaik berdasarkan nilai *fitness* (besar *shipping fee* yang diperoleh) dari populasi saat ini agar dapat dijadikan induk (*parent*) untuk generasi berikutnya.
  - **Persilangan (*Crossover*):** Menggabungkan materi genetik dari dua induk terpilih untuk menghasilkan keturunan baru yang mewarisi sifat-sifat baik dari kedua induknya.
  - **Mutasi (*Mutation*):** Dilakukan dengan probabilitas tertentu untuk mengubah secara acak sebagian kecil atribut kromosom (seperti mengubah posisi koordinat x, y atau membalikkan orientasi kendaraan). Hal ini bertujuan untuk mempertahankan keragaman populasi dan keluar dari jebakan *local optima*.

- · **Mekanisme *Elitism* / candidate_peak**

Untuk memastikan bahwa solusi terbaik yang pernah ditemukan selama proses pencarian tidak hilang akibat proses *crossover* atau mutasi yang merusak, program menerapkan mekanisme *elitism*. Variabel penampung candidate_peak secara konsisten memantau dan menyimpan individu dengan nilai *fitness* tertinggi (max_fitness) di setiap generasi. Dengan cara ini, nilai terbaik yang telah dicapai akan terus dipertahankan hingga akhir iterasi.

C. Hasil Eksperimen

1. Hill-Climbing with Sideways Move

Eksperimen dimulai dengan menjalankan terminal dengan kode berikut untuk membuka folder algoritma berada:

```
cd path/…/src
```

Kemudian jalankan program ini untuk memulai search:

```
python run.py
```

Setelah beberapa detik, algoritma akan mengembalikan data hasil dari pencarian.

Algoritma menggunakan data berikut:

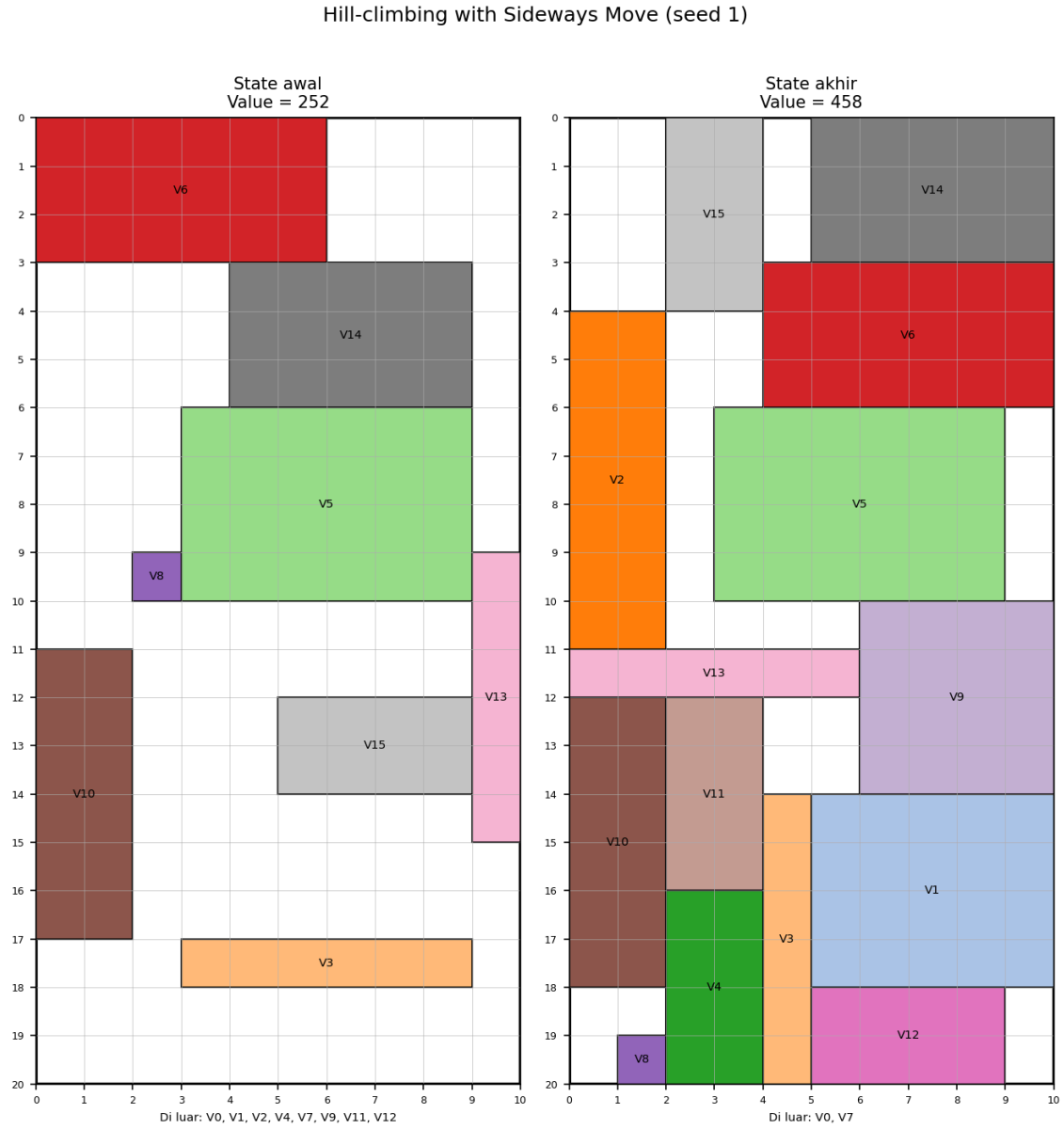

```
{
  "ship":
  {"w": 10, "l": 20, "max_capacity": 400},

  "vehicles": [
    {"id": "V0", "w": 4, "l": 5, "fee": 15, "weight": 21, "eta": 9},
    {"id": "V1", "w": 4, "l": 5, "fee": 48, "weight": 35, "eta": 6},
    {"id": "V2", "w": 2, "l": 7, "fee": 27, "weight": 23, "eta": 3},
    {"id": "V3", "w": 1, "l": 6, "fee": 15, "weight": 39, "eta": 10},
    {"id": "V4", "w": 2, "l": 4, "fee": 22, "weight": 9, "eta": 6},
    {"id": "V5", "w": 4, "l": 6, "fee": 52, "weight": 27, "eta": 7},
    {"id": "V6", "w": 3, "l": 6, "fee": 46, "weight": 40, "eta": 4},
    {"id": "V7", "w": 7, "l": 7, "fee": 280, "weight": 200, "eta": 9},
    {"id": "V8", "w": 1, "l": 1, "fee": 10, "weight": 5, "eta": 10},
    {"id": "V9", "w": 4, "l": 4, "fee": 41, "weight": 25, "eta": 2},
    {"id": "V10", "w": 2, "l": 6, "fee": 38, "weight": 20, "eta": 3},
    {"id": "V11", "w": 4, "l": 2, "fee": 20, "weight": 25, "eta": 9},
    {"id": "V12", "w": 4, "l": 2, "fee": 48, "weight": 40, "eta": 5},
    {"id": "V13", "w": 1, "l": 6, "fee": 32, "weight": 39, "eta": 4},
    {"id": "V14", "w": 3, "l": 5, "fee": 26, "weight": 29, "eta": 6},
    {"id": "V15", "w": 2, "l": 4, "fee": 33, "weight": 17, "eta": 3}
  ]
}
```


Berikut hasil dari percobaan:

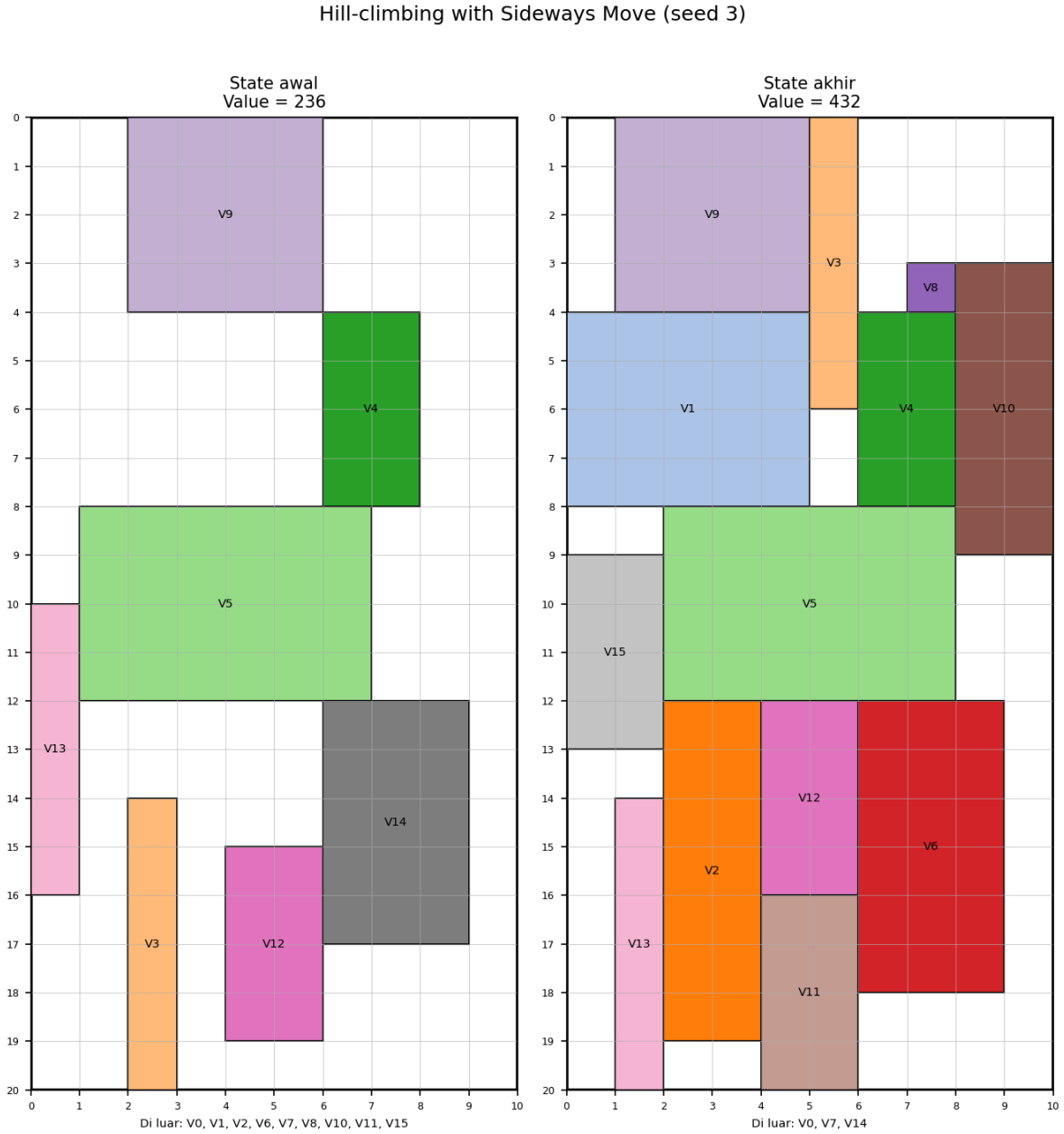
Percobaan 1 (seed 1):

State awal (value = 252): Kendaraan di dalam : V3, V5, V6, V8, V10, V13, V14, V15
State akhir (value = 458): Kendaraan di dalam: V1, V2, V3, V4, V5, V6, V8, V9, V10, V11, V12, V13, V14, V15
Durasi eksperimen : 5,75 detik
Berhenti di : max_sideways
Jumlah iterasi : 86



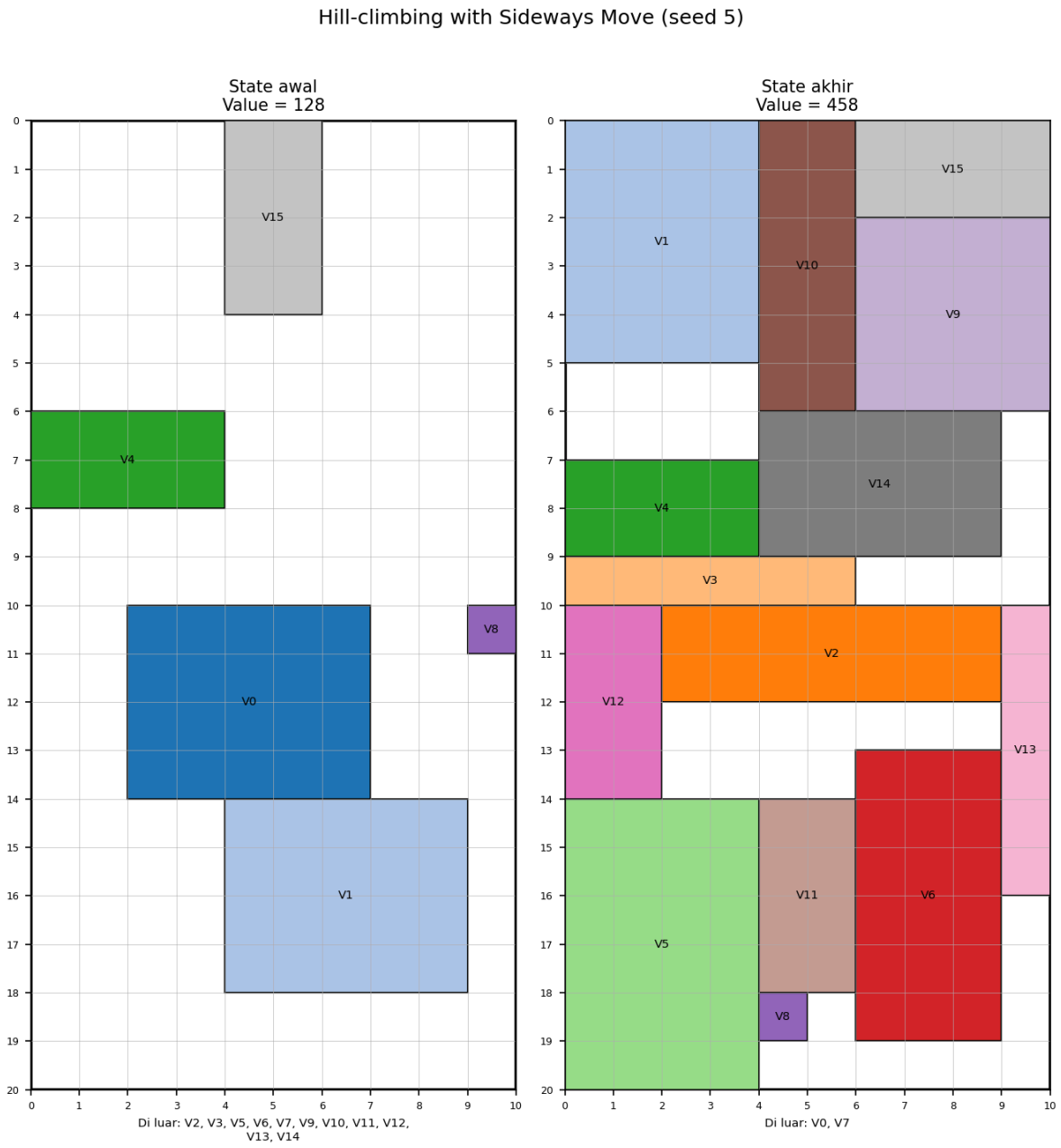
Percobaan 2 (seed 3):

State awal (value = 236): Kendaraan di dalam : V3, V4, V5, V9, V12, V13, V14
State akhir (value = 432): Kendaraan di dalam: V1, V2, V3, V4, V5, V6, V8, V9, V10, V11, V12, V13, V15
Durasi eksperimen : 7,04 detik
Berhenti di : max_sideways
Jumlah iterasi : 39



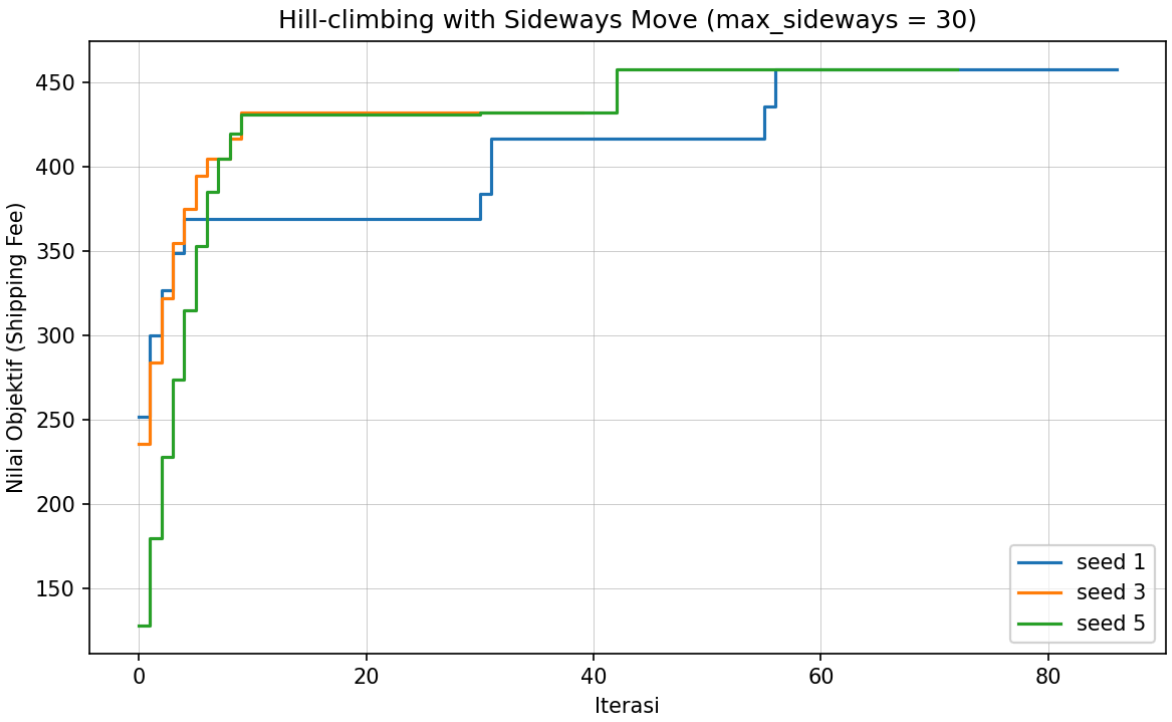
Percobaan 3 (seed 5):

State awal (value = 128): Kendaraan di dalam : V0, V1, V4, V8, V15
State akhir (value = 458): Kendaraan di dalam: V1, V2, V3, V4, V5, V6, V8, V9, V10, V11, V12, V13, V14, V15
Durasi eksperimen : 16,1 detik
Berhenti di : max_sideways
Jumlah iterasi : 72



Perbandingan Nilai Objektif Setiap Iterasi

Visualisasi ini memberikan ringkasan antara 3 percobaan yang dilakukan. Grafik ini menunjukkan nilai objektif pada setiap iterasi. Dapat diketahui pada grafik ini tentang jumlah iterasi per percobaan, pertumbuhan nilai objektif dan langkah *sideways*-nya, dan nilai *local optimum* yang di dapat.



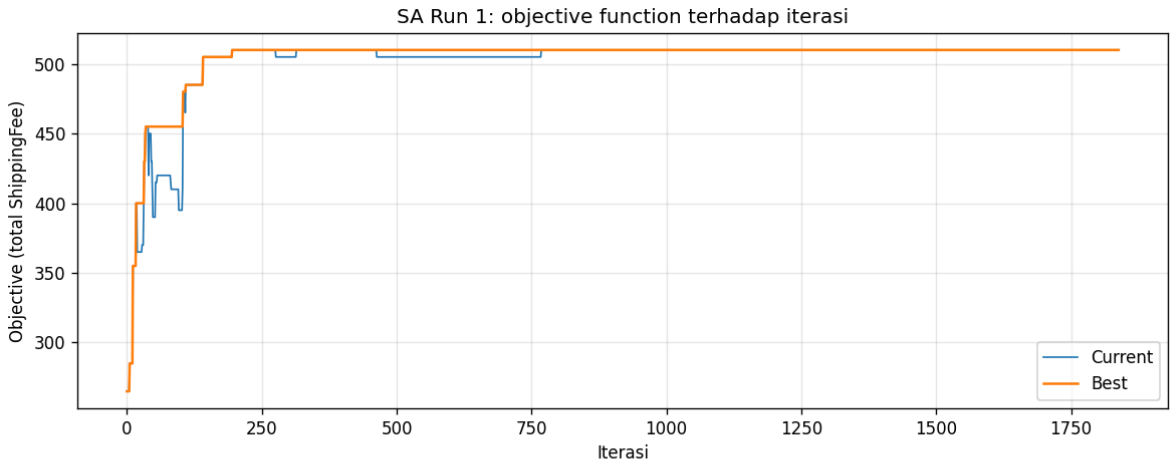
2. Simulated Annealing

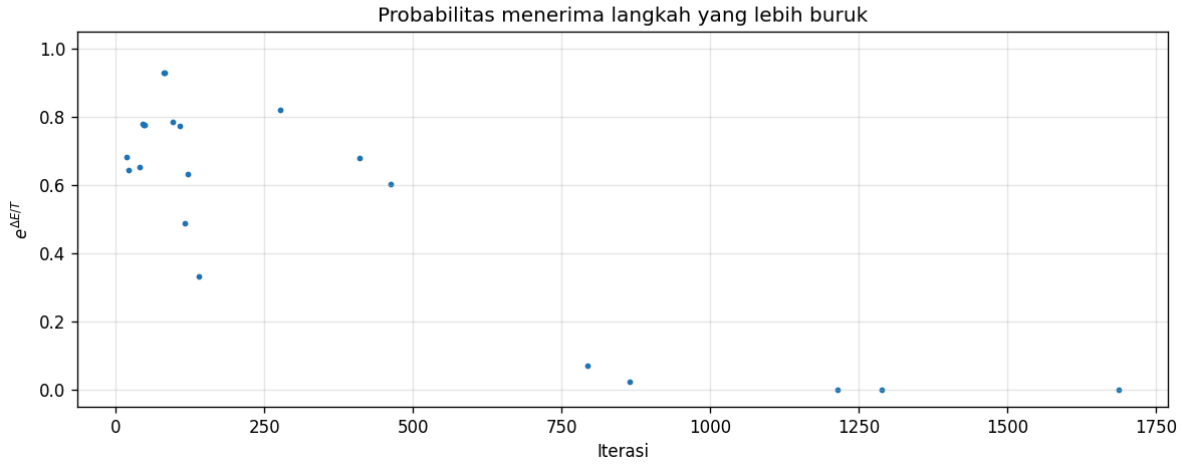
Percobaan 1:
State awal (value = 265): Di dalam : A=V1, B=V3, C=V6, D=V7, E=V9, F=V12
Di luar : V2, V4, V5, V8, V10, V11

State akhir (value = 510): Di dalam : A=V1, B=V3, C=V4, D=V5, E=V6, F=V7,
G=V8,H=V9, I=V10, J=V11, K=V12
Di luar : V2

H = Horizontal, V = Vertical

Total berat : 248 / 250
Valid : True
Iterasi : 1838
Durasi : 0.144 detik
Frekuensi stuck : 16
Langkah buruk diterima: 11





Percobaan 2:

State awal (value = 230): Di dalam : A=V1, B=V4, C=V9, D=V10, E=V12

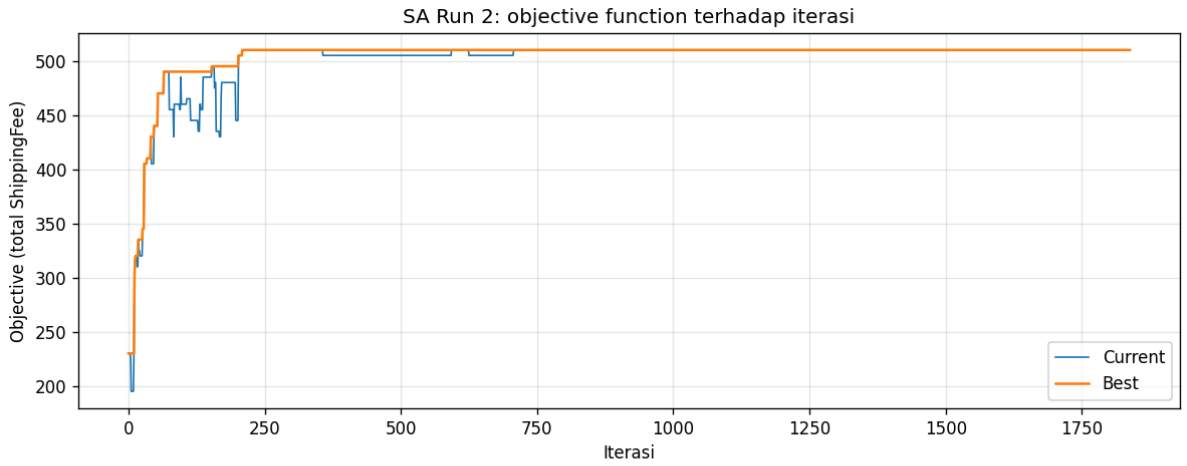
Di luar : V2, V3, V5, V6, V7, V8, V11

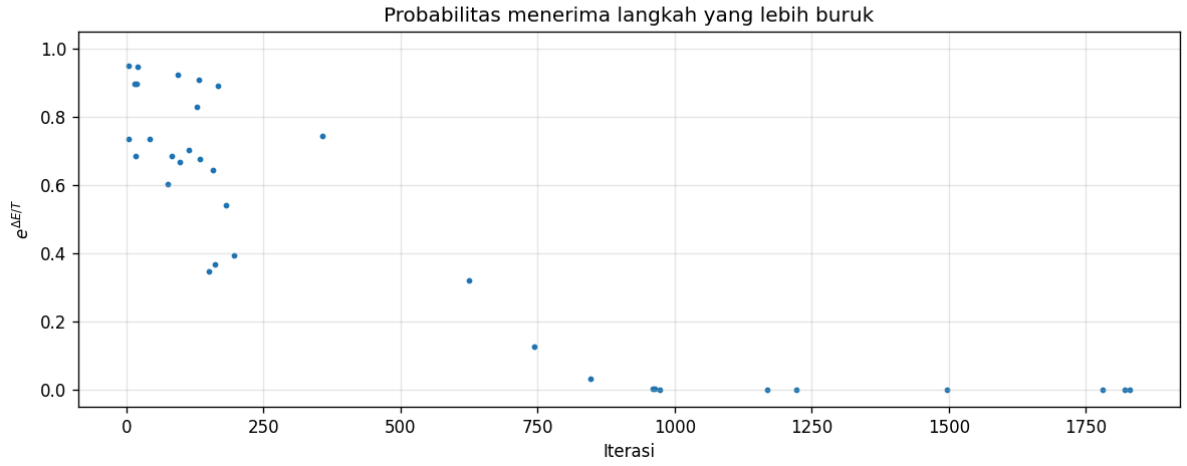
State akhir (value = 510): Di dalam : A=V1, B=V3, C=V4, D=V5, E=V6, F=V7, G=V8, H=V9, I=V10, J=V11, K=V12

Di luar : V2

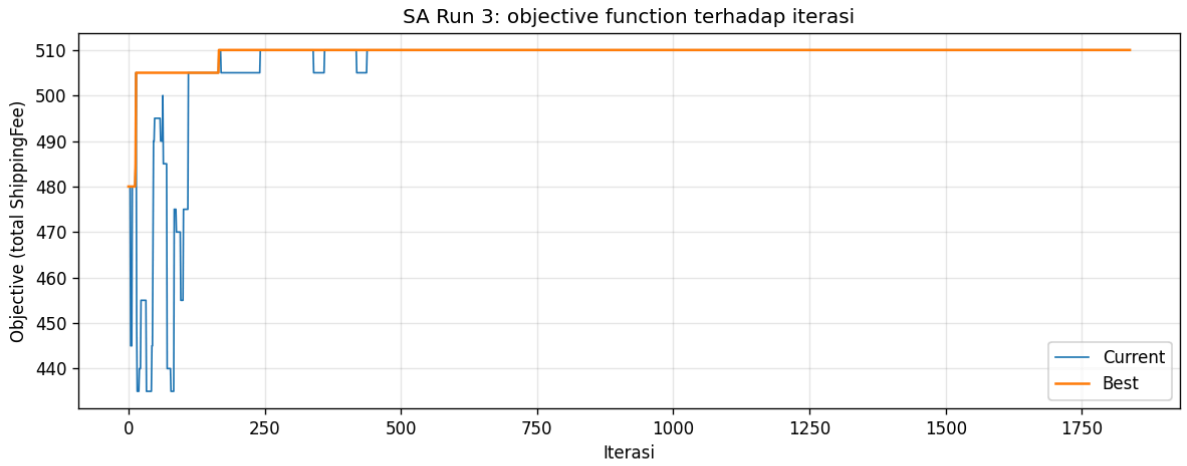
H = Horizontal, V = Vertical

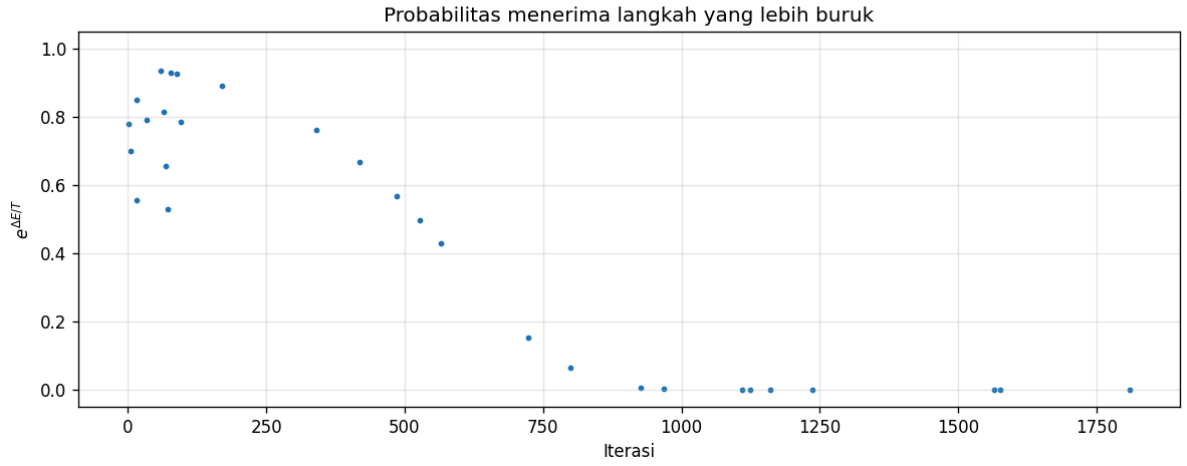
Total berat : 248 / 250
Valid : True
Iterasi : 1838
Durasi : 0.153 detik
Frekuensi stuck : 16
Langkah buruk diterima: 19





Percobaan 3:
State awal (value = 480): Di dalam : A=V1, B=V2, C=V4, D=V5, E=V6, F=V9, G=V10, H=V11, I=V12
Di luar : V3, V7, V8
State akhir (value = 510): Di dalam : A=V1, B=V3, C=V4, D=V5, E=V6, F=V7, G=V8, H=V9, I=V10, J=V11, K=V12
Di luar : V2
Total berat : 248 / 250
Valid : True
Iterasi : 1838
Durasi : 0.133 detik
Frekuensi stuck : 17
Langkah buruk diterima: 13





Simulated Annealing dijalankan sebanyak tiga kali. Pada run 1, nilai awal sebesar 265 meningkat menjadi 510. Nilai 510 pertama kali dicapai pada iterasi ke-195, dengan durasi pencarian 0,118 detik dan frekuensi stuck sebanyak 16 kali. Pada run 2, nilai awal sebesar 230 meningkat menjadi 510 pada iterasi ke-209, dengan durasi 0,194 detik dan frekuensi stuck sebanyak 16 kali. Pada run 3, nilai awal sebesar 480 meningkat menjadi 510 pada iterasi ke-166, dengan durasi 0,165 detik dan frekuensi stuck sebanyak 17 kali.

Ketiga run sama-sama berhenti setelah 1.838 iterasi, yaitu saat suhu turun di bawah suhu minimum. Pada ketiga state akhir, total berat kendaraan yang diangkut adalah 248 dari kapasitas 250, dan semua kendaraan berhasil dimasukkan ke dalam kapal kecuali V2.

3. Genetic Algorithm

Kelompok 1: Jumlah Iterasi = 30 (Populasi = 30)

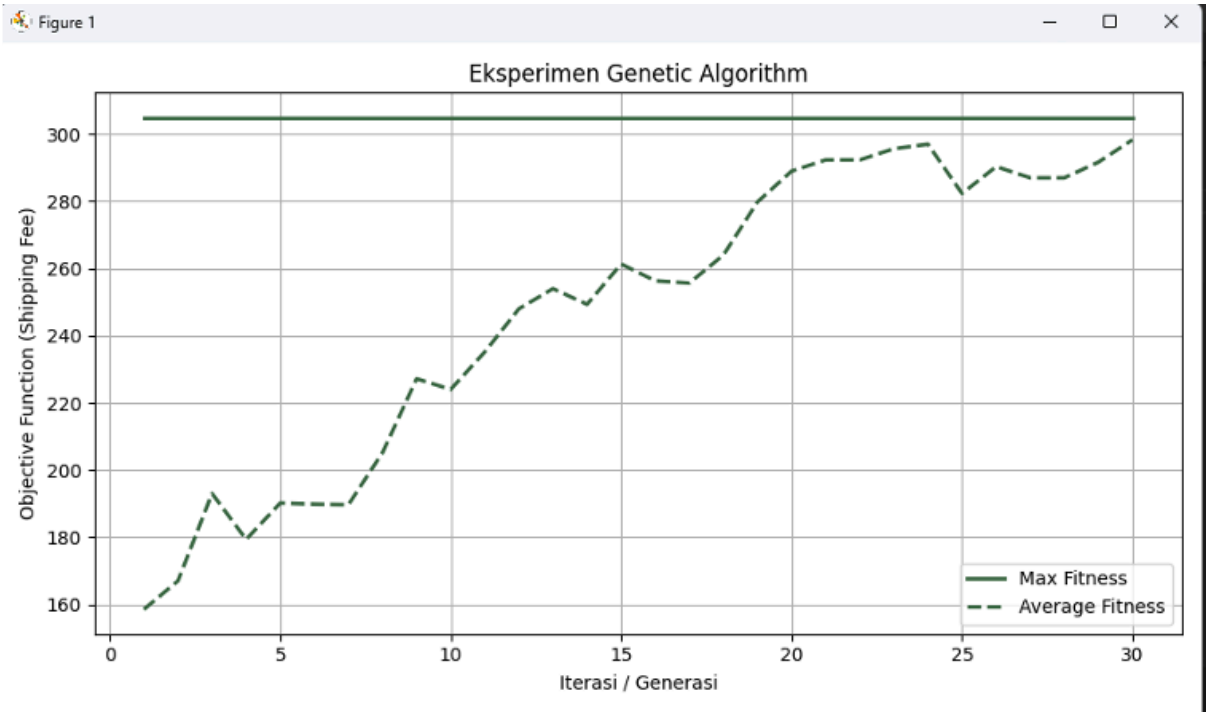
1. **Percobaan 1** :

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 30

f. Durasi Eksekusi: 0.32 detik

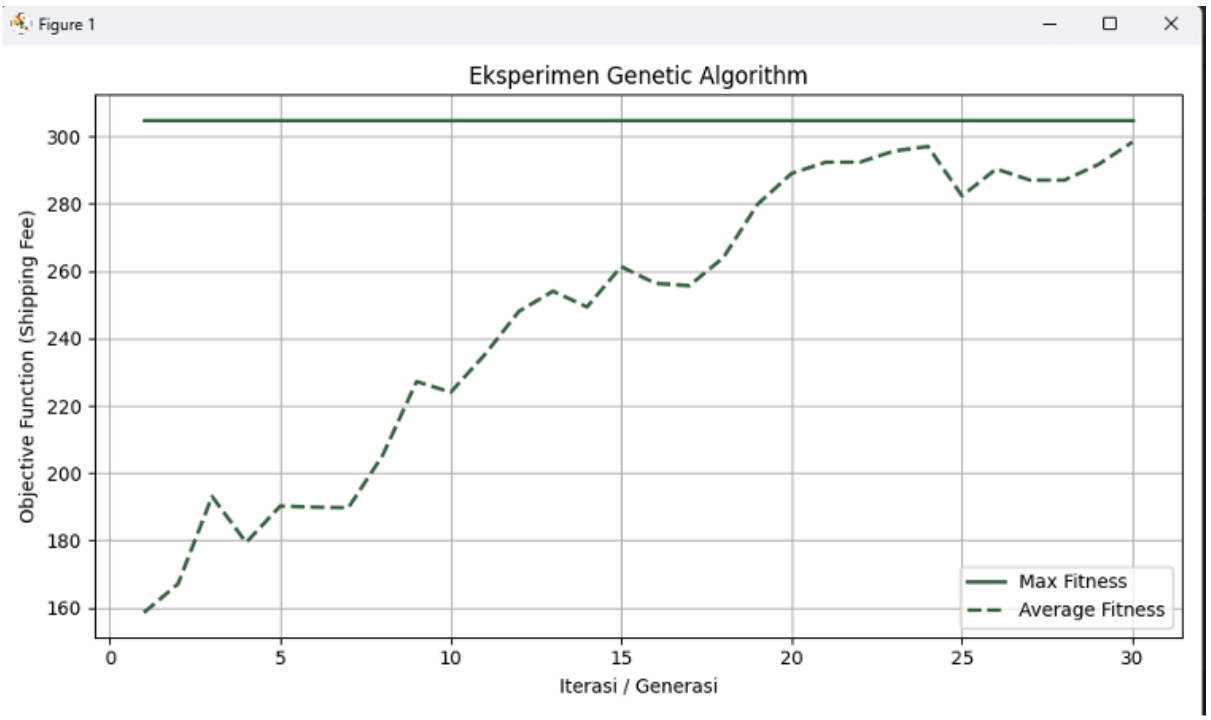
2. **Percobaan 2**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 30

f. Durasi Eksekusi: 0.22 detik

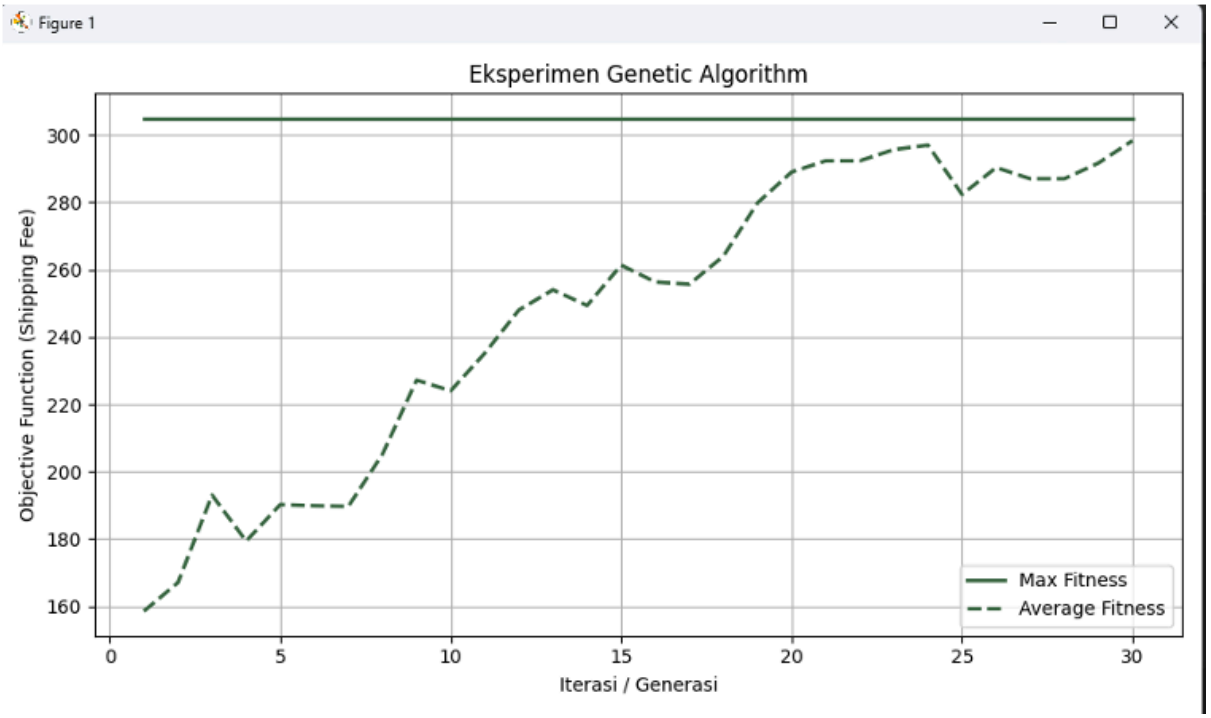
3. **Percobaan 3**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 30

f. Durasi Eksekusi: 0.39 detik

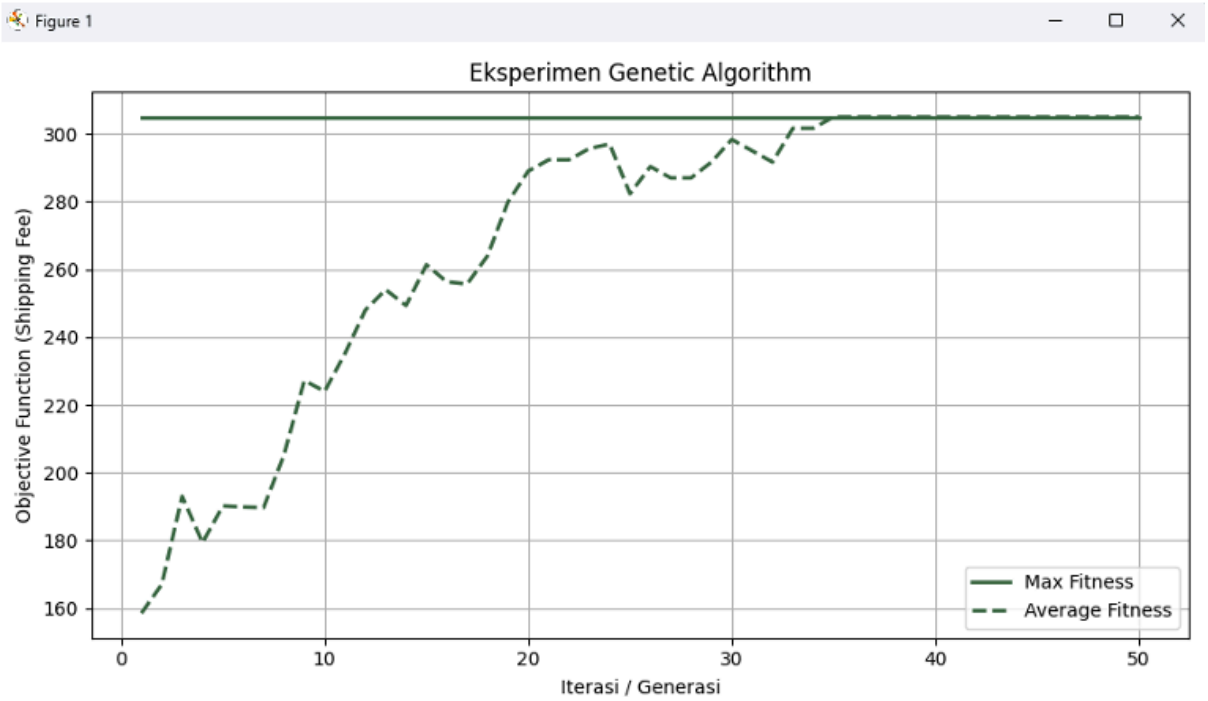
Kelompok 2: Jumlah Iterasi = 50 (Populasi = 30)

1. **Percobaan 1**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')
State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 50

f. Durasi Eksekusi: 0.43 detik

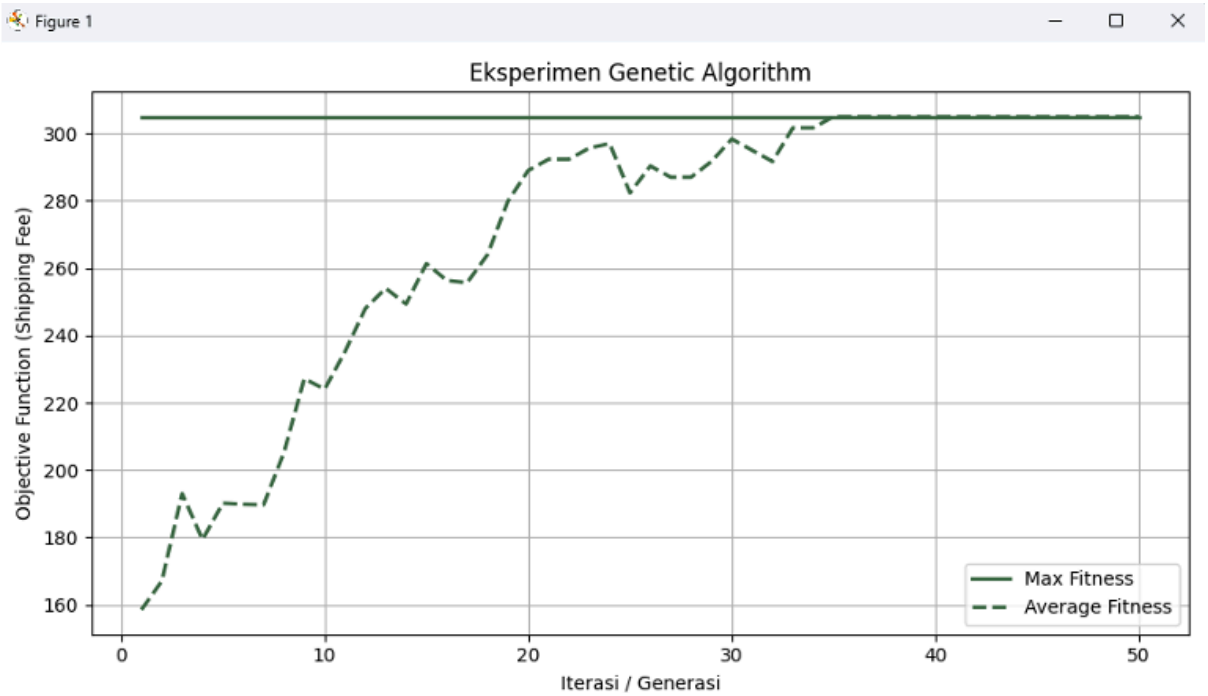
2. **Percobaan 2**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 50

f. Durasi Eksekusi: 0.41 detik

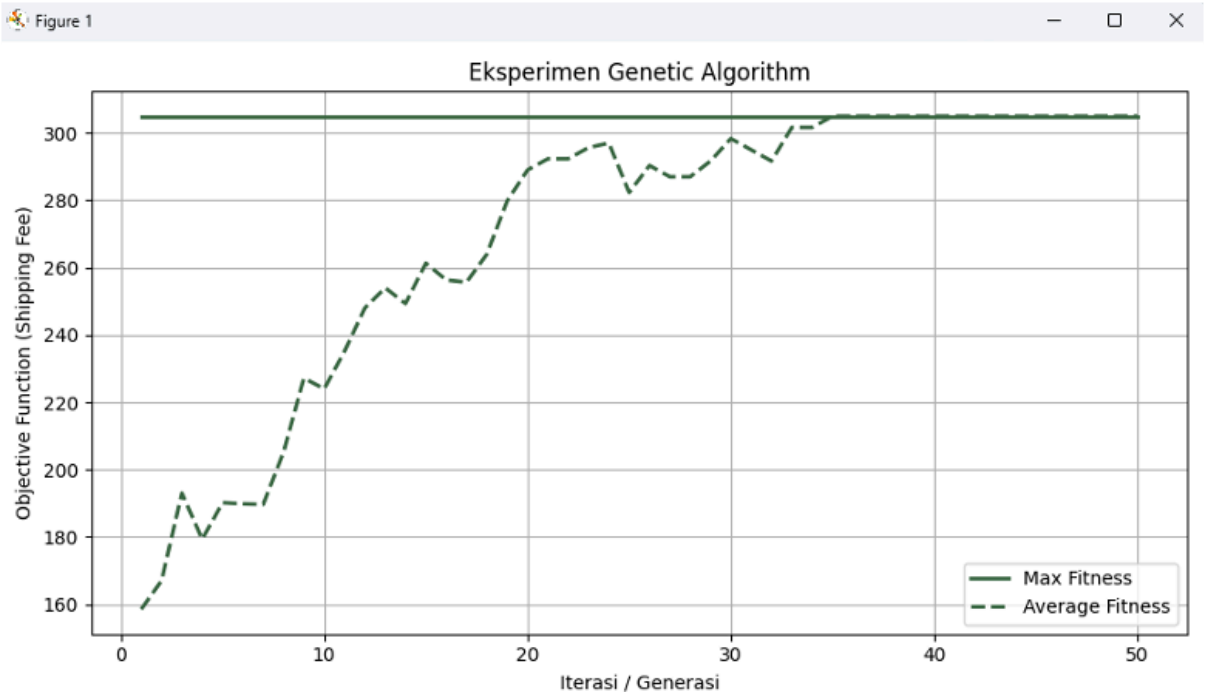
3. **Percobaan 3**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 50

f. Durasi Eksekusi: 0.44 detik

Kelompok 3: Jumlah Iterasi = 100 (Populasi = 30)

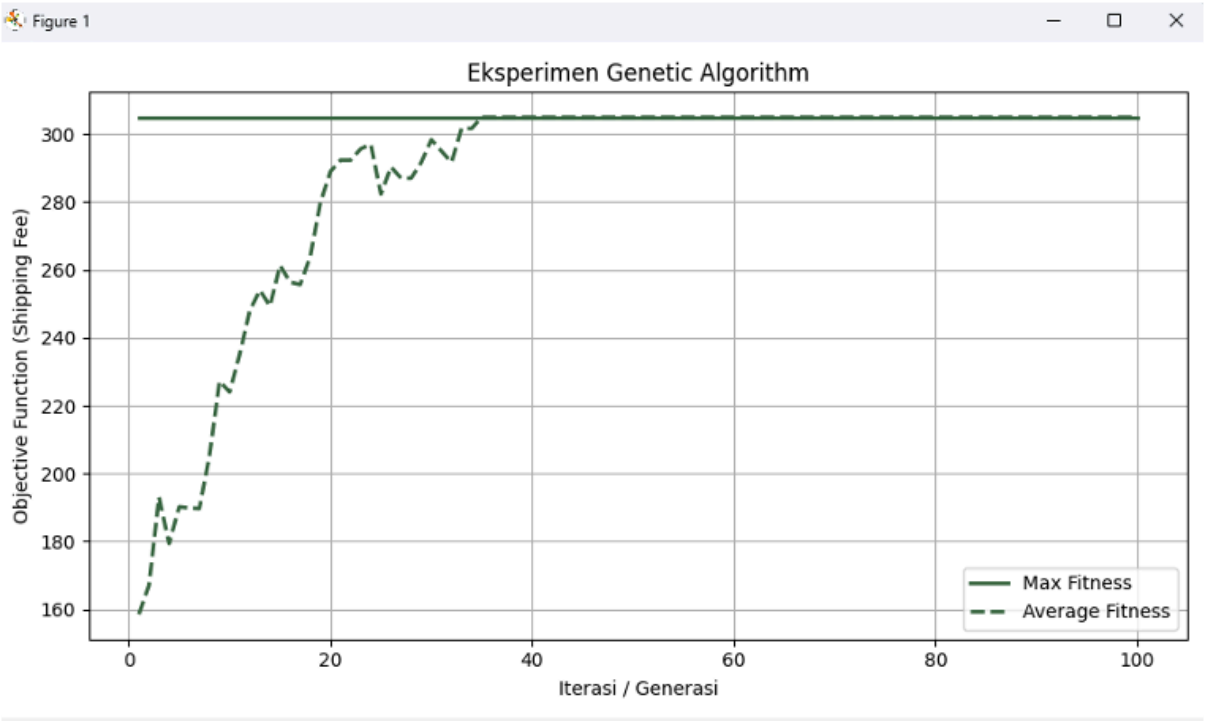
1. **Percobaan 1**:

a. State Awal & Akhir: a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 100

f. Durasi Eksekusi: 0.68 detik

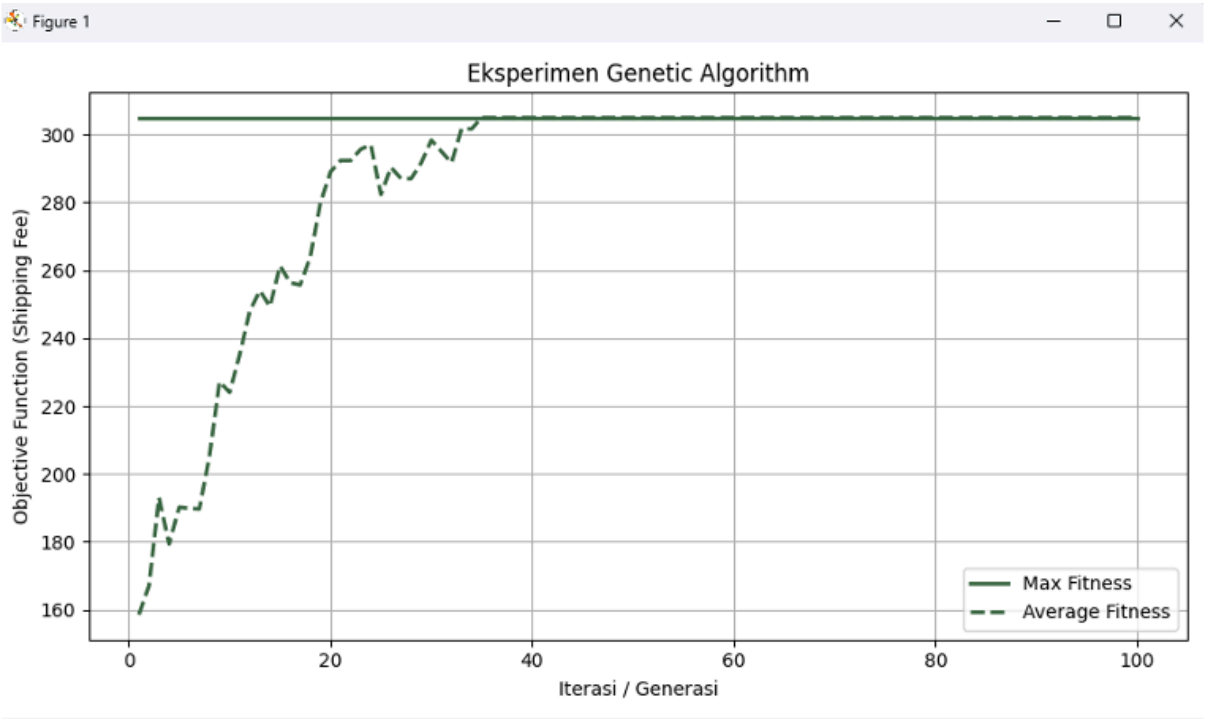
2. **Percobaan 2**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 100

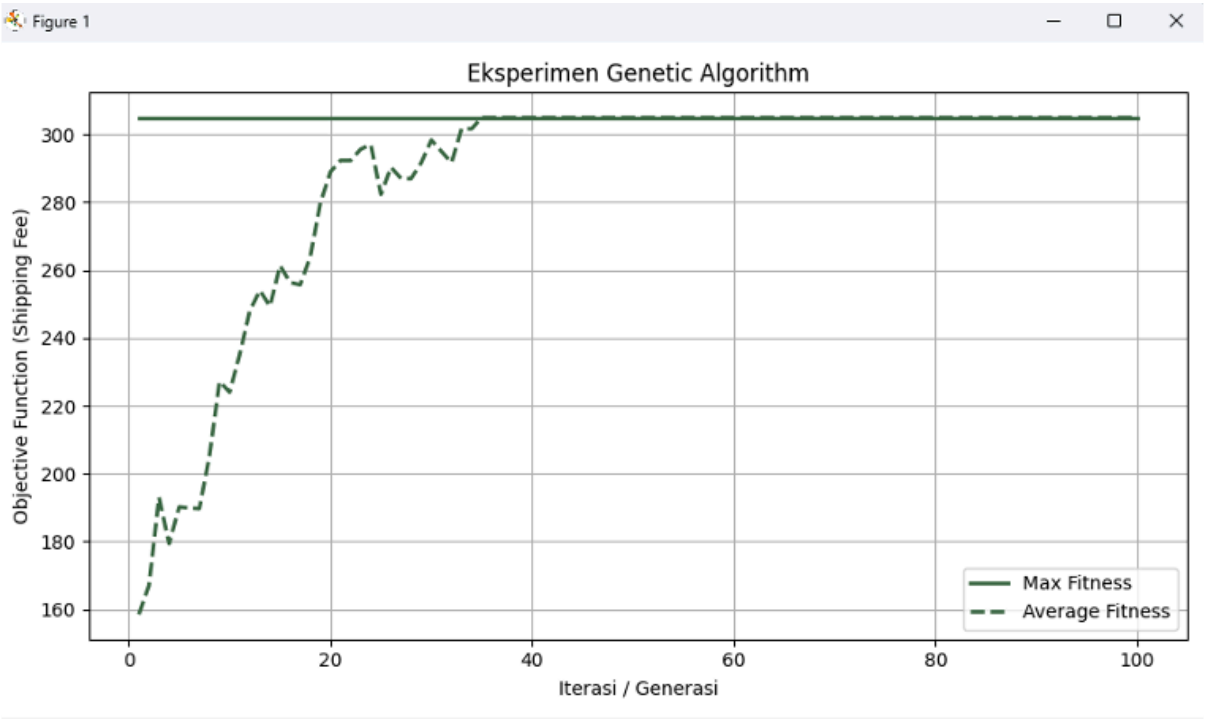
f. Durasi Eksekusi: 1.05 detik

3. **Percobaan 3**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')
State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 100

f. Durasi Eksekusi: 1.33 detik

Kelompok Pengujian: Ukuran Populasi = 5 (Iterasi = 50)

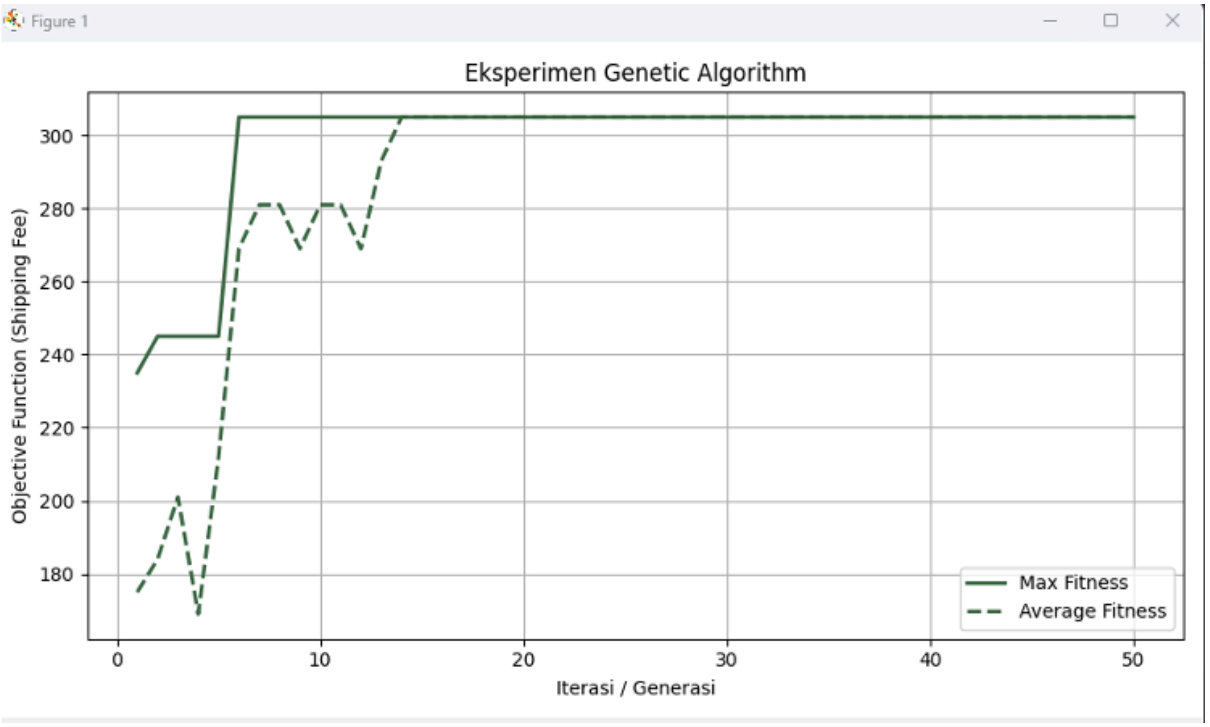
1. **Percobaan 1**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=0, y=11 (Horizontal), V2: x=1, y=12 (Horizontal), V3: x=4, y=6 (Vertical), V4: x=3, y=14 (Vertical), V5: x=1, y=12 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 5

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.06 detik

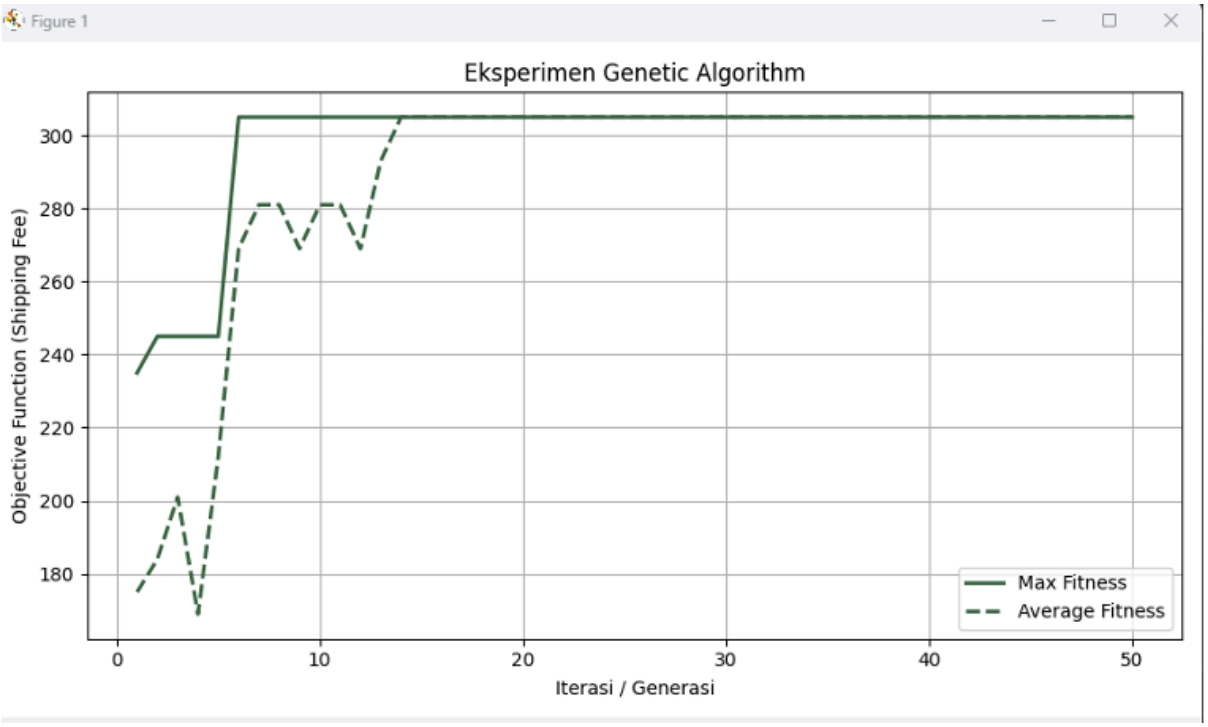
2. **Percobaan 2**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=0, y=11 (Horizontal), V2: x=1, y=12 (Horizontal), V3: x=4, y=6 (Vertical), V4: x=3, y=14 (Vertical), V5: x=1, y=12 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 5

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.04 detik

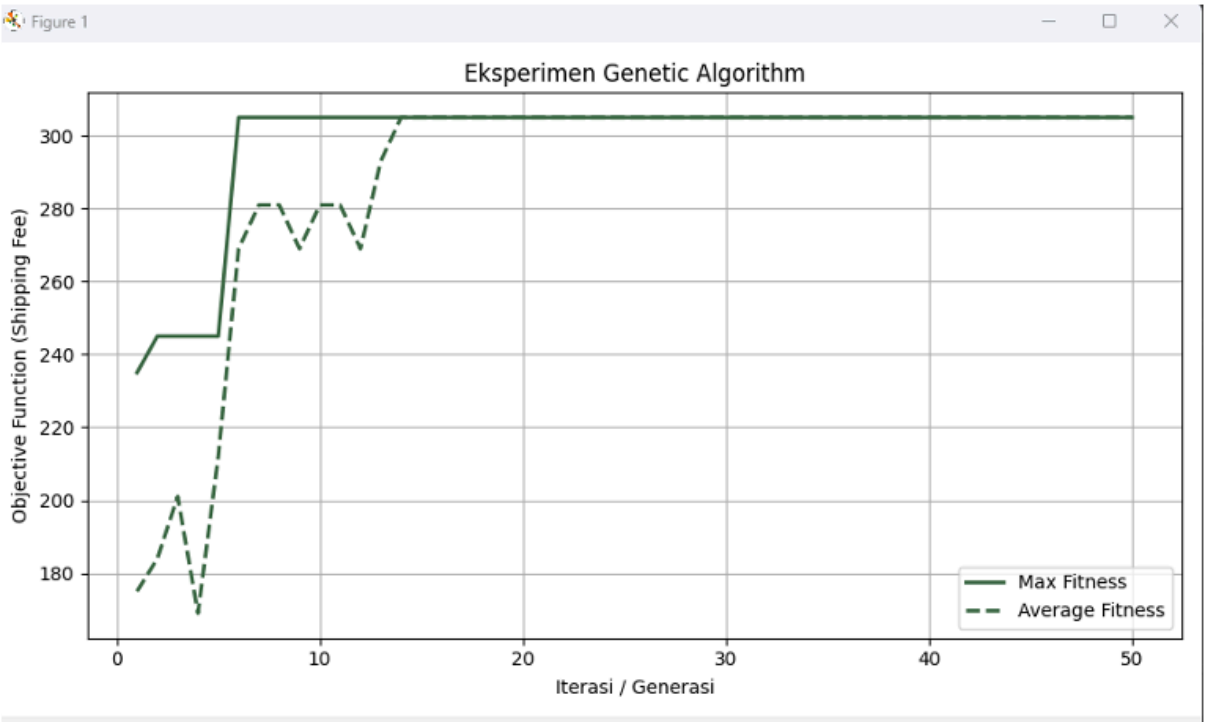
3. **Percobaan 3**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=0, y=11 (Horizontal), V2: x=1, y=12 (Horizontal), V3: x=4, y=6 (Vertical), V4: x=3, y=14 (Vertical), V5: x=1, y=12 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 5

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.05 detik

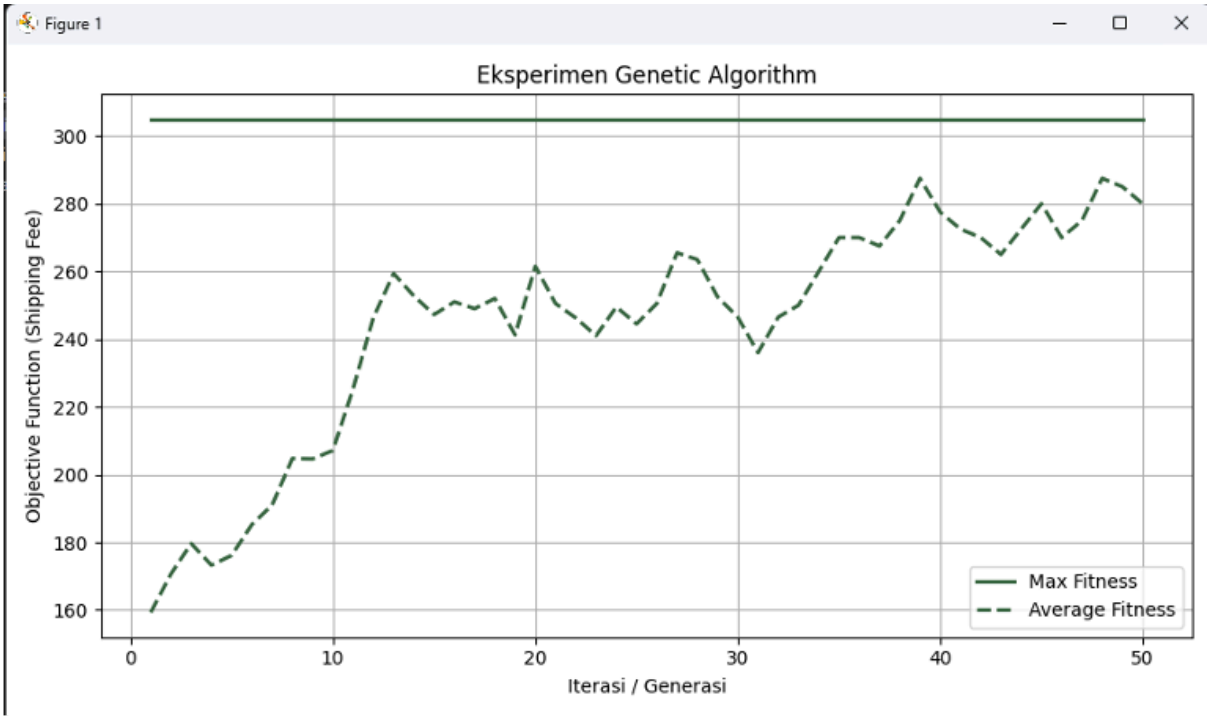
Kelompok Pengujian: Ukuran Populasi = 40 (Iterasi = 50)

1. **Percobaan 1**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')
State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 40

e. Banyak Iterasi: 50

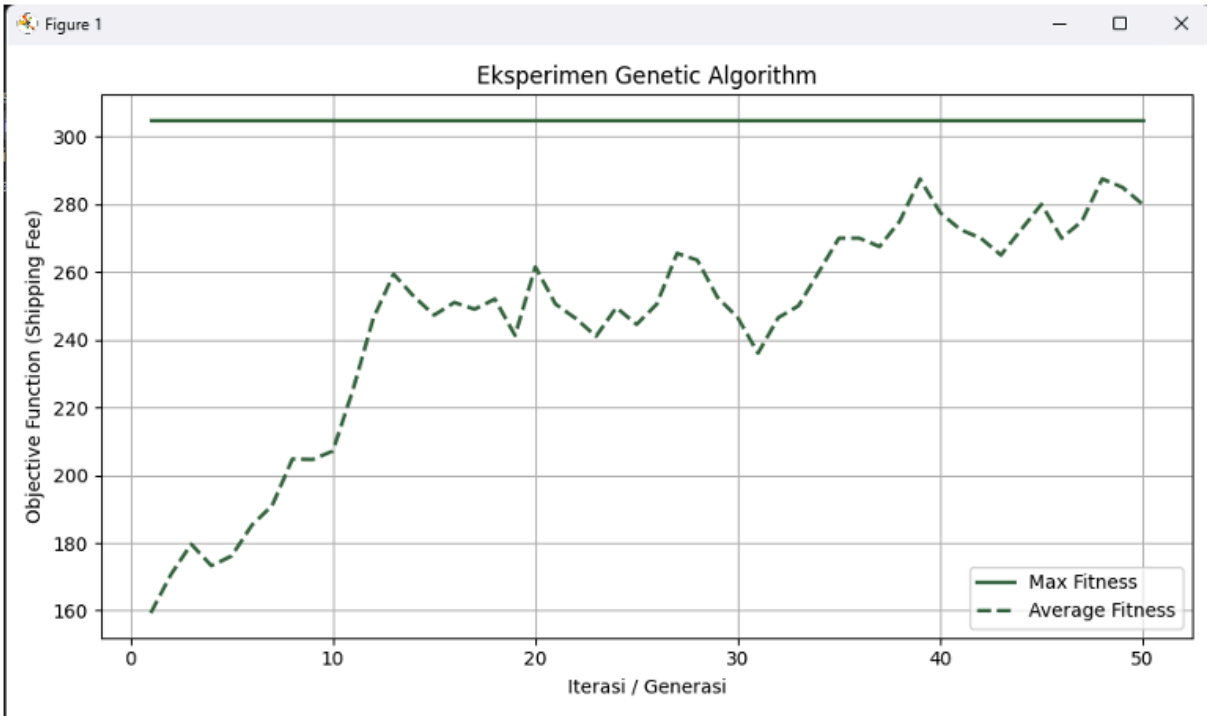
f. Durasi Proses Pencarian: 0.43 detik

2. **Percobaan 2**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')
State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 40

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.77 detik

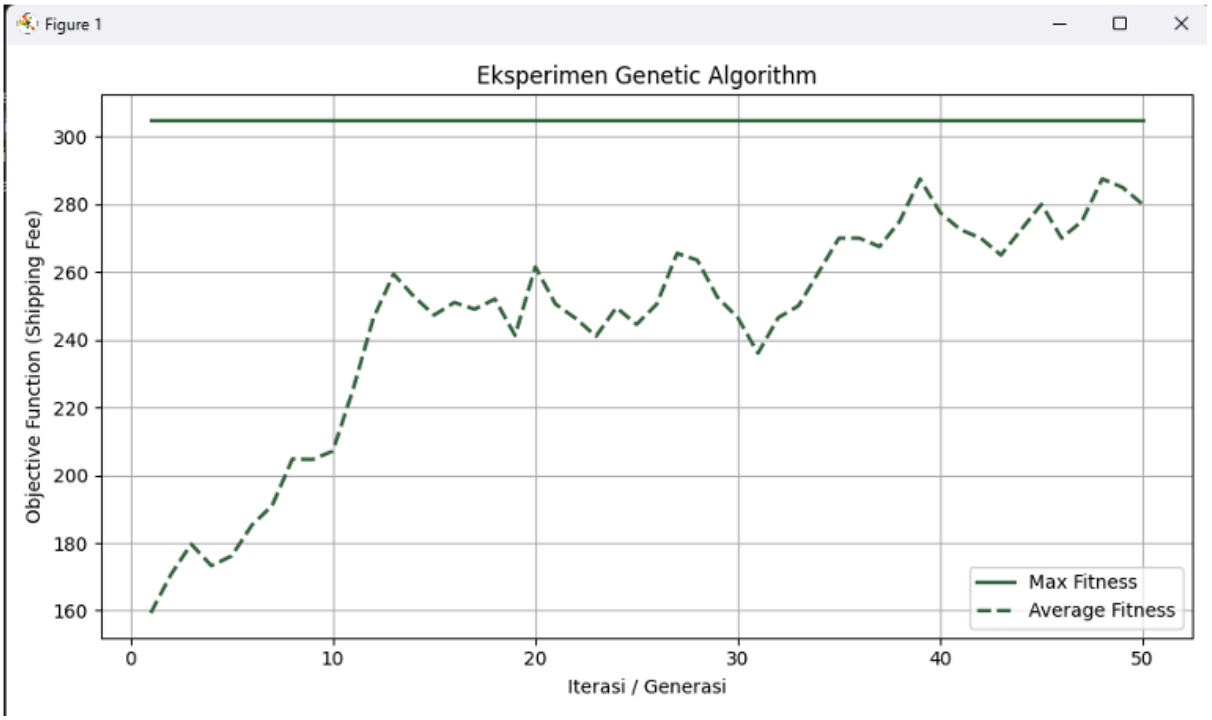
3. **Percobaan 3**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 40

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.62 detik

Kelompok Pengujian: Ukuran Populasi = 30 (Iterasi = 50)

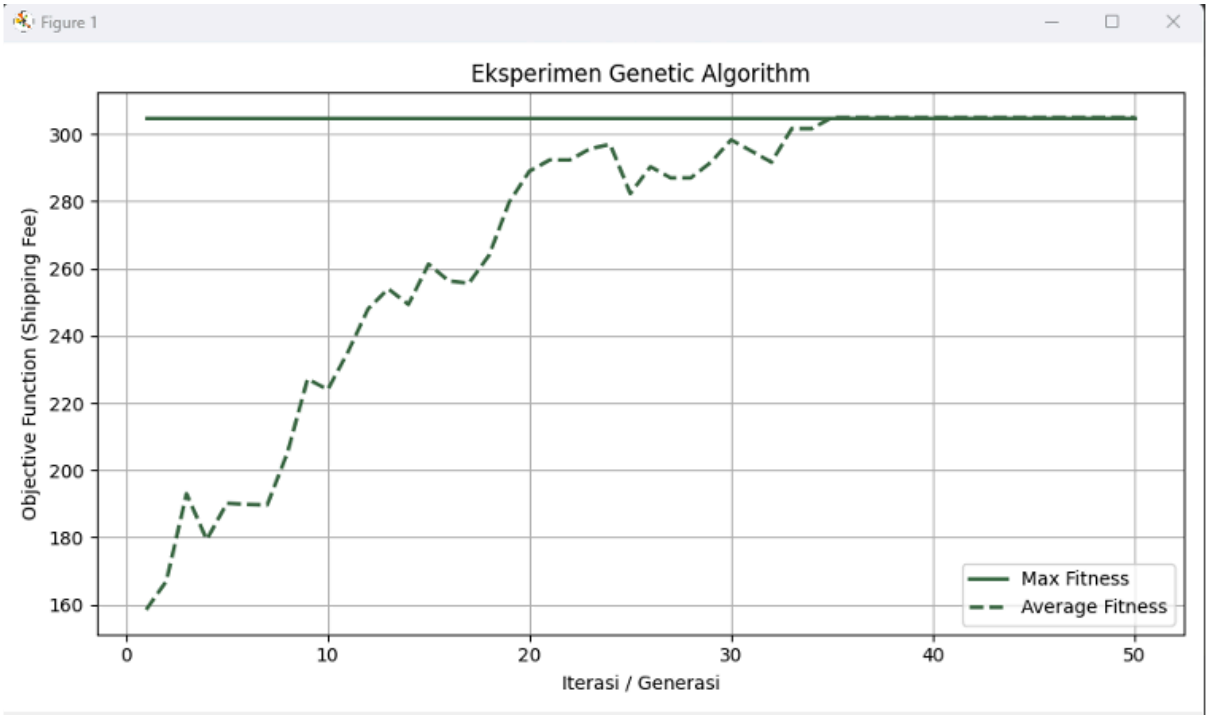
1. **Percobaan 1**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.37 detik

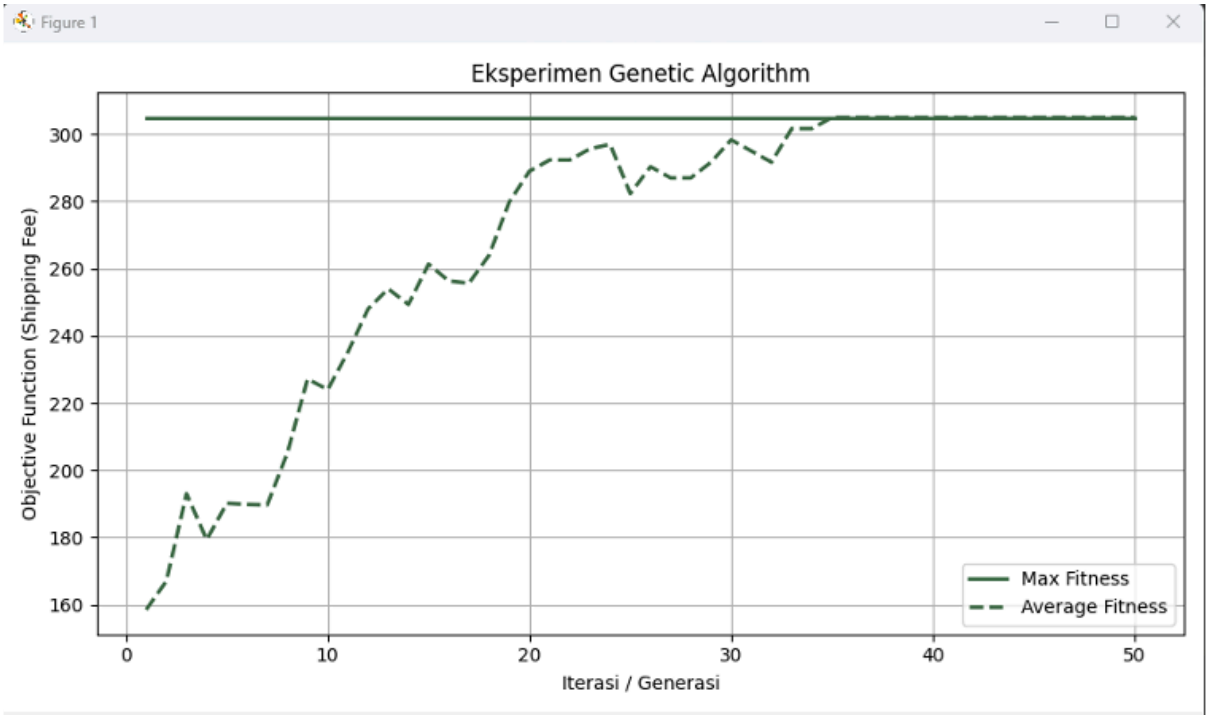
2. **Percobaan 2**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30

e. Banyak Iterasi: 50

f. Durasi Proses Pencarian: 0.38 detik

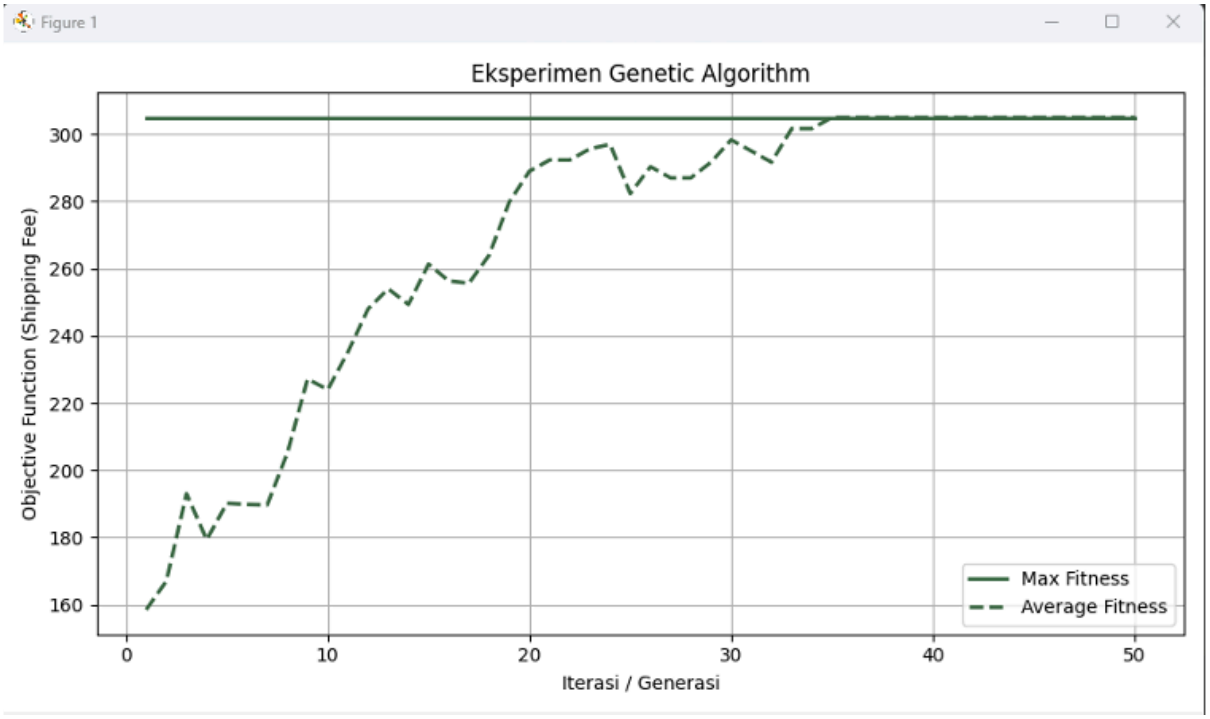
3. **Percobaan 3**:

a. State Awal: V1: None, V2: None, V3: None, V4: None, V5: Placement(x=6, y=1, orientation='Horizontal')

State Akhir: V1: x=5, y=7 (Vertical), V2: x=7, y=2 (Horizontal), V3: x=3, y=14 (Vertical), V4: x=2, y=5 (Horizontal), V5: x=0, y=8 (Vertical)

b. Objective Function Akhir: 305

c. Plot Grafik:



d. Jumlah Populasi: 30
e. Banyak Iterasi: 50
f. Durasi Proses Pencarian: 0.37 detik

D. Analisis dan Kesimpulan

- **Hill-Climbing with Sideways Move**
  Algoritma ini berhasil meningkatkan total *Shipping Fee* secara signifikan dari *state* awal. Pada 3 percobaan, nilai *state* awalnya berturut-turut 252, 236, dan 128. Setelah eksperimennya, nilainya meningkat menjadi 458, 432, dan 458. Waktu yang digunakan tiap percobaan adalah 5,75; 7,04; dan 16,1 detik. Durasi ini relatif lambat karena banyaknya evaluasi nilai *neighbor*

Hasil akhir yang didapat cukup konsisten antara ketiga eksperimen, dengan selisih yang cukup kecil. Ini menandakan bahwa algoritma relatif tidak sensitif terhadap *state* awal.

Ketiga eksperimen berhenti karena mencapai batas *max sideways move*, yang mana artinya pencarian terjebak di *local optimum.* Ini menandakan bahwa pencarian masih belum tentu berada di puncak global.

- **Genetic Algorithm**
  - Seluruh pengujian *Genetic Algorithm* berhasil konsisten mencapai nilai optimal 305 (karena ukuran *search space* yang relatif kecil dengan inisialisasi populasi yang memadai).
  - Populasi sebesar (30, 40) memperkaya keragaman genetik dan membuat konvergensi rata-rata populasi lebih stabil, sementara populasi kecil (5) membuat eksekusi jauh lebih cepat.
  - Iterasi yang lebih tinggi (50–100) memberikan waktu bagi seluruh populasi untuk menyatu (*converge*) dengan individu terbaik.
  - Durasi eksekusi program yang tergolong sangat cepat (di bawah 1.5 detik).

E. Pembagian Tugas

Padmanabha Adyaksa (18222142)	Membuat genetic algorithm dan melakukan tes
Zafran Abqary Faisal (18225084)	Membuat Hill-Climbing Search with Sideways Move dan melakukan tes
Fawwaz Aydin Mustofa (18222109)	Membuat simulated annealing dan melakukan tes

F. Referensi

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley Publishing Company.

G. Lampiran (Form Penggunaan Generative AI)

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:
Nama/NIM : Zafran Abqary Faisal/18225084
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelegensi Artifisial
Judul Tugas/Proposal : IMPLEMENTASI ALGORITMA LOCAL SEARCH DALAM OPTIMASI PENEMPATAN KENDARAAN DALAM KAPAL

Menyatakan bahwa ~~saya~~/kami* ~~menggunakan/~~tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	~~Ya~~/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	~~Ya~~/Tidak*	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	
Lainnya, Jelaskan:	~~Ya~~/Tidak*	

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada
mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Zafran Abqary Faisal
18225084